From Theory to Practice: Oblivious RAM and Its Role in Modern Secure Memory Systems

Sourajit Karmakar
Department of Computer Science
The Pennyslvania State University
State College, United States
sourajit@psu.edu

***Abstract*—Memory access patterns always have the possibility of leaking sensitive information even when the content of the underlying data is encrypted. This critical vulnerability poses a serious threat in systems that rely on untrusted memory infrastructures. Oblivious Random Access Memory addresses this challenge by ensuring that physical memory access patterns are independent of a program's logical behavior. This survey demonstrates a systematic overview of ORAM and its modern applications. It outlines the threat models and formal definitions that form the foundation of oblivious memory systems. The paper further examines classical ORAM constructions and their progression toward more efficient and specialized designs, with particular attention to performance and storage tradeoffs. In addition, the application of ORAM in modern systems such as trusted execution environments, secure processors, and privacy preserving computation frameworks is analyzed. In this paper, the motivation is the practical limitations that hinder widespread deployment, including latency overhead and energy consumption, and highlight open research challenges in the study of ORAM systems.**

Keywords—Oblivious RAM, ORAM, secure memory, access-pattern leakage, trusted execution environments, side-channel attacks

I. INTRODUCTION

Modern computing has been increasingly relying on shared and often untrusted memory. These include everyday media like cloud storage, a website allocating memory on your computer, memory access from a sandboxed process, and more. While common encryption protocols and safety measures in place protect the content of memory, it does not prevent leakage through access patterns. Observing which memory locations are accessed by a program, or the order in which read/write operations occur, can tell an adversary information that might compromise the end-user's privacy and security even with encryption in place [1], [6]. Such side-channel attacks could pose serious risks in systems where memory is controlled by untrusted software or infrastructure.

Oblivious Random Access Memory (ORAM) addresses this vulnerability by ensuring that the sequence of physical memory access is statistically independent of the program's logical behavior [1]. The novel work by Goldreich and Ostrovsky formalized what ORAM really is and what it is capable of through demonstration of its abilities to protect memory access patterns under well-defined threat models [1]. However, early ORAM constructions provided strong theoretical guarantees but incurred significant performance overheads [2].

Subsequent research primarily focused on addressing the efficiency and practicality of shortcomings. There are many types of constructions I discuss in this survey paper like hierarchical ORAM constructions and tree-based designs, such as Path ORAM and Ring ORAM, which not only help maintain obliviousness but also maintaining obliviousness [3], [4], [5]. Additional variants, including write-only ORAM [9], read-only ORAM [8], and integrity-preserving ORAM [7], further optimized ORAM for specific workloads and threat scenarios.

Slowly but steadily, the adoption of trusted execution environments (TEEs) has become more widespread through technologies such as Intel Software Guard Extensions (SGX), AMD Secure Encrypted Virtualization (SEV), Google Titan M, and Apple Secure Enclave. This increased adoption has rejuvenated research interest in ORAM. Even though TEEs are able to protect enclave memory contents from untrusted operating systems, they still remain vulnerable to access-pattern side channels [15], [16], [18]. Modern systems have integrated ORAM and ORAM-inspired techniques to secure enclaves, cloud databases, and privacy-preserving computation frameworks [15], [16], [17], [18], [19]. Hardware-assisted ORAM designs that integrate obliviousness directly into memory controllers have also been proposed to reduce performance overhead [20], [21].

However, despite these advances, deployment of ORAM has remained a bit limited mainly due to the same reasons; latency overhead and implementation complexity. This survey provides a comprehensive overview of ORAM, examining classical constructions, modern optimizations, and system-level applications. I also discuss practical limitations and open research challenges in balancing security guarantees with performance and the ability to be deployed easily.

II. BACKGROUND AND THREAT MODEL

A. Memory Systems Overview

Modern computing systems differentiate logical memory, which is a program's view of the memory layout and physical memory where the data is actually stored on the hardware. As mentioned before, encryption protects the contents of physical memory from unauthorized reading or writing, but it does not prevent information leakage through access patterns. The sequence of read and write operations can reveal sensitive information to an adversary, potentially

compromising security, as demonstrated in [1], [6]. This is the motivation that led to the design of ORAM, where access patterns are randomized or obscured to prevent inference by an adversary.

B. Threat Model

Security by obscurity is not a solution. This is why ORAM operates under a clearly defined adversary model where multiple groups of adversaries are considered. A clear understanding of the threat model is critical, as ORAM does not aim to protect against all possible attacks, but rather against information leakage through memory access patterns. The threat models can be broken into the following categories:

1) Honest-but-Curious Adversary: When Goldreich and Ostrovsky originally introduced ORAM, it was introduced under the "honest-but-curious" adversary model. In this model, the adversary executes the protocol correctly while passively observing all interactions with memory. This type of adversary is common in the world of cloud computing where the storage provider is not automatically assumed to compromise the integrity of the system or the data in it intentionally but may attempt to infer sensitive information from observable behavior [1], [15]. Systems such as data-oblivious file systems for Intel SGX also adopt this threat model, assuming the underlying storage layer is curious but non-malicious [15].

2) Malicious System Adversary: Next, a stronger and more realistic adversary is considered. This is particularly relevant in modern secure processors and TEEs. In this model, the adversary controls the operating system, hypervisor, or memory controller, while the application executes inside a protected enclave [16], [17], [18], [19]. This adversary may observe all physical memory accesses, reorder or delay memory responses, trigger page faults or interrupts, exploit microarchitectural side channels, but won't be able to break through cryptographic primitives and directly read enclave registers. Research has shown that TEEs such as Intel SGX are highly vulnerable to access-pattern leakage, even when memory contents are encrypted [16], [18]. As a result, ORAM is frequently proposed as a complementary defense, ensuring that enclave memory access remains oblivious to the untrusted system software [17], [19].

3) Side-Channel Observers: Side-channel observers are potential adversaries that can monitor sensitive information from secondary effects of memory access behavior. Such side channels include timing variations caused by differing memory access patterns, cache-based leakage where hits and misses reveal access locality, observation of memory bus transactions, and DRAM-level effects such as row buffer conflicts or bank activation patterns. For something like timing variations, even very small differences can show a large effect that can help the adversary be in an advantageous spot, motivating the design of fully oblivious memory access protocols that eliminate data-dependent access behavior [18], [19].

Based on Table 1, it can be seen all adversaries have some amount of control over the host system and ORAM poses an important use to protect access pattern information.

TABLE 1. Access Matrix For Threat Models

Model Type	Type of Access Granted			
	Observational	*Data R/W*	*Controls OS*	*Timing*
Honest-but-Curious	Yes	No	No	Yes
Malicious System	Yes	Yes	Yes	Yes
Side-Channel	Yes	No	Yes/No	Yes

C. ORAM Security Definitions

The oblivious aspect of ORAM ensures that an adversary observing the memory system cannot infer any information about a program's logical memory access behavior. The ORAM scheme is considered oblivious if any two logical access sequences of equal length exist, then the corresponding physical access sequences observed by the adversary are computationally indistinguishable to anyone observing the memory patterns. This definition guarantees that access-pattern leakage is eliminated regardless of the underlying data values or control-flow decisions, assuming standard cryptographic hardness assumptions hold [1], [2]. As a result, ORAM protects against a wide range of adversaries that can monitor memory access but cannot break cryptographic primitives.

To evaluate ORAM constructions in practice, several key performance and security metrics are commonly used. One of the most critical metrics is bandwidth overhead, defined as the ratio of physical memory accesses to logical memory accesses. Early hierarchical ORAM constructions incurred polylogarithmic bandwidth overheads due to repeated reshuffling and rebuilding operations, making them impractical for latency-sensitive applications [2]. Subsequent designs improved bandwidth efficiency through more structured data layouts and access strategies, as seen in tree-based ORAM schemes such as Path ORAM which significantly reduce the number of physical accesses per logical operation while maintaining obliviousness [3], [4].

III. Classical ORAM Constructions

Oblivious RAM research began with theoretical constructions aimed at hiding memory access patterns while maintaining correctness and efficiency. Classical ORAM constructions laid the foundation for practical implementations by defining the core techniques for reshuffling, data obliviousness, and secure storage.

A. Hierarchical ORAM

The earliest ORAM construction introduced by Goldreich and Ostrovsky [1] is the hierarchical ORAM, which organize memory into multiple levels, each exponentially larger than the previous. Logical data blocks are stored in these levels, and access involves reshuffling and re-encrypting blocks to prevent an adversary from linking repeated accesses to the same logical data. Hierarchical ORAM achieves computational obliviousness but suffers from significant bandwidth overhead, as each logical access requires multiple reads and writes across levels [2], [3]. Although the original construction is mostly of theoretical interest, it provided the formal basis for subsequent

Pseudocode 1: ORAM Data Structures

```
class ObliviousStack: # Simple conceptual program to understand oram()
    def __init__(self, oram):
        self.oram = oram
        self.size = 0
    def push(self, value):
        self.size += 1
        self.oram.write(self.size, value)  # Access pattern hidden by ORAM
    def pop(self):
        if self.size == 0:
            return None
        value = self.oram.read(self.size)
        self.oram.write(self.size, None)  # Obliviously erase
        self.size -= 1
        return value
```

optimizations and motivated tree-based schemes that significantly reduce bandwidth and latency [3], [4].

$$L = O(\log(N_{mb})),\ N_{mb} \in N \quad (1)$$

From Equation (1), N_{mb} represents hierarchical levels, each containing exponentially more blocks than the previous. Over time, hierarchical ORAM has proven to be secure against honest-but-curious adversaries but due to it's high latency and large number of reshuffles per acces, it is not a preferred solution for practical applications.

B. Square Root ORAM

An intermediate construction, known as square-root ORAM**,** was introduced to reduce the overhead of hierarchical ORAM. In this scheme, memory is split into √N-sized buckets, and logical accesses are randomized and reshuffled in these smaller units [2]. Square-root ORAM reduces the number of levels and lowers the reshuffling cost compared to fully hierarchical ORAM. This leads to moderate reductions in bandwidth and latency while maintaining the same formal security guarantees. However, this approach still incurs non-negligible overhead, making it less suitable for high-performance systems or environments with tight latency constraints.

C. Tree-Based ORAM

One of the most practical ORAM constructions was the tree-based construction, particularly Path ORAM, introduced by Stefanov et al. [3], [4]. In this design, the untrusted server organizes memory blocks in a binary tree, with each block assigned a random leaf node just like how I see in Fig 1.

Each logical access involves reading and rewriting the entire path from the root to the assigned leaf, including dummy blocks to maintain indistinguishability. Path ORAM significantly reduces bandwidth overhead and client storage compared to hierarchical ORAM by storing only a small stash and position map on the client. Optimized variants, such as Optimal Oblivious RAM, further reduce computational and communication costs while maintaining strong security guarantees [4].

D. Oblvious Data Structures (ODSs)

ODSs extend the principles of ORAM to common data abstractions such as arrays, stacks, queues, and search trees, enabling structured operations without leaking access patterns. While ORAM ensures that individual memory accesses are oblivious, ODSs guarantee that high-level operations such as inserting an element into a priority queue or searching in a map do not reveal information about the data being manipulated [1], [3], [4]. These data structures are particularly important in secure processors, trusted execution environments, and cloud-based applications, where adversaries can observe multiple operations and infer the given sensitive information from patterns of reads and writes [15], [16].

A simple example of an ODS is an oblivious stack implemented using Path ORAM. In this construction, each push or pop operation is translated into a constant number of ORAM read and write operations, ensuring that every stack operation accesses a full root-to-leaf path in the ORAM tree. As a result, the physical memory access pattern remains indistinguishable from random accesses, regardless of the logical stack behavior. Pseudo-code 1 illustrates a conceptual implementation of an oblivious stack built atop an ORAM interface. Here, oram.read and oram.write operations abstract away the underlying physical memory accesses, which are made oblivious by the ORAM protocol. Even though the stack size changes dynamically, each operation generates an access trace that is computationally indistinguishable from any other sequence of the same length, satisfying the formal definition of obliviousness [1], [3]. This approach can be extended to more complex data structures such as queues, maps, and priority queues, forming the foundation of oblivious computation in modern secure systems [4], [15], [16].

IV. ORAM IN MODERN SYSTEMS

While ORAM was initially proposed as a theoretical primitive, its relevance has grown significantly with the emergence of modern systems that rely on untrusted memory infrastructures**.** In practice, many contemporary platforms protect data using encryption but still leak sensitive information through memory access patterns.

A. Trusted Execution Environments (TEEs)

TEEs such as Intel SGX and AMD SEV aim to protect application code and data from an untrusted operating system by executing sensitive workloads inside hardware-isolated enclaves. While these platforms encrypt memory contents and protect registers, they do not inherently hide memory access patterns, making them vulnerable to access-pattern side-channel attacks [16], [18].

Extensive research has demonstrated that enclave memory accesses such as page faults, cache activity, and memory bus transactions can be observed by a malicious operating system or hypervisor. These observations allow attackers to infer control flow, data-dependent branches, and even cryptographic secrets, despite strong cryptographic protections on memory contents [16], [18]. In Intel SGX, for example, page-fault side channels and cache-based attacks have been shown to leak sensitive application behavior with high accuracy [18].

ORAM is frequently proposed as a complementary defense to TEEs, addressing this fundamental limitation by ensuring that enclave memory access is oblivious. By enforcing that every logical memory access results in an indistinguishable physical access pattern, ORAM prevents the operating system from learning any information beyond the length of the execution trace [17], [19]. Several SGX-based systems integrate ORAM to protect enclave-resident data structures, enabling secure execution even in the presence of a fully compromised OS [15], [17].

However, deploying ORAM within TEEs introduces nontrivial overhead. The limited enclave memory size, increased latency due to enclave transitions, and constrained memory bandwidth exacerbates ORAM's inherent performance costs. As a result, recent work focuses on lightweight ORAM variants and hybrid designs that balance security with practicality in enclave-based systems [15], [19].

B. Secure Processors

Aside from the present TEEs, ORAM has also been incorporated into the design of secure processors, where memory confidentiality and access-pattern privacy are treated as primary architectural steppingstones. In these systems, ORAM functionality is implemented directly in hardware, typically within the memory controller or cache hierarchy, to reduce the performance overhead associated with software-based ORAM [20], [21].

Hardware-assisted ORAM designs exploit parallelism, pipelining, and dedicated on-chip buffers to accelerate ORAM operations. By moving ORAM logic closer to memory, secure processors can reduce latency and amortize bandwidth overhead across multiple requests [20]. These approaches are particularly effective for tree-based ORAM schemes such as Path ORAM, where deterministic access patterns enable efficient hardware scheduling [3], [21].

Secure processors that have integrated ORAM have demonstrated good protection against adversaries capable of observing all off-chip memory traffic. This model is especially relevant in cloud and multi-tenant environments, where physical access to memory buses or shared memory resources cannot be ruled out [20], [21]. However, some of the bigger challenges still remain with the increased hardware complexity and die-size area cost for silicon that are used in ORAM-enabled processors, limiting their adoption to high-security domains.

C. Privacy-Preserving Computation

ORAM also plays a critical role in privacy-preserving computation frameworks, including secure databases and secure multi-party computation (MPC). In these settings, data confidentiality alone is insufficient, as query patterns, access frequency, and control flow may reveal sensitive information about the underlying dataset or user intent [1], [15].

In secure database systems, ORAM is used to hide which records are accessed during queries, preventing leakage of access locality and query structure. By combining ORAM with encrypted storage, these systems ensure that both data contents and access patterns remain private, even when hosted on untrusted servers [15]. Similarly, ORAM serves as a foundational building block in MPC protocols, where participants jointly compute a function over private inputs without revealing intermediate memory access behavior [1], [16].

Oblivious data structures that are used with ORAM such as oblivious maps, priority queues, and search trees help enable complex and secure computation while also being able to preserve the privacy guarantees [3], [4]. ORAM does introduce additional computational and bandwidth overhead, they are often the only viable mechanism for achieving full access-pattern privacy in adversarial environments.

V. Simulated Demonstration

An experimental evaluation of a functional Path ORAM prototype implemented in software to demonstrate access-pattern obliviousness and quantify the associated performance overheads. The goal of this evaluation is not to benchmark a production-ready ORAM system, but rather to empirically illustrate the core security and efficiency tradeoffs discussed throughout this survey.

A. Experimental Setup

I implemented a simplified Path ORAM model in Python that captures the essential components of tree-based ORAM schemes, including a binary tree layout, a position map, a client-side stash, and full root-to-leaf path accesses for each logical operation. Cryptographic protections such as encryption and message authentication codes were intentionally not added here in an effort to isolate and analyze access-pattern behavior. The ORAM tree stores N logical blocks, organized as a complete binary tree with bucket size Z = 4 blocks per node. Each logical access performs a read and write along a randomly assigned root-to-leaf path, as specified by the Path ORAM protocol [3]. Experiments were conducted on varying values of N to observe scalability trends.

B. Trace-Based Side-Channel Analysis

To evaluate access-pattern obliviousness, I compared memory access traces generated by ORAM-protected accesses against those produced by a conventional memory model. In the non-oblivious baseline, repeated access to the same logical address produces identical physical access traces which are able to expose the access locality. Using Fig. 1, it is evident that the ORAM implementation successfully mitigates this vulnerability by generating access traces that consist of randomly selected root-to-leaf paths, even when the same logical address is accessed multiple times consecutively. As illustrated by the randomized distribution of physical path IDs in Fig. 1, this decoupling of logical behavior from physical memory

locations confirms that Path ORAM effectively hides access locality.

In contrast, the ORAM implementation generates access traces that consist of randomly selected root-to-leaf paths, even when the same logical address is accessed multiple times consecutively. This demonstrates ORAM's effectiveness in mitigating trace-based side-channel attacks, such as those targeting page faults or cache activity in trusted execution environments [16], [18].

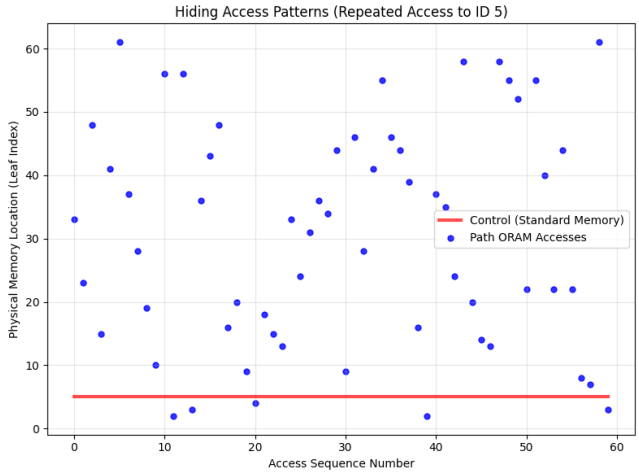


Fig. 1. Trace-based side-channel analysis comparing physical memory access patterns.

C. Bandwidth Overhead

I measured bandwidth overhead as the ratio of physical memory accesses to logical memory accesses. For each logical access, the ORAM implementation performs a full traversal of a tree path of height O(log N), resulting in multiple physical reads and writes.

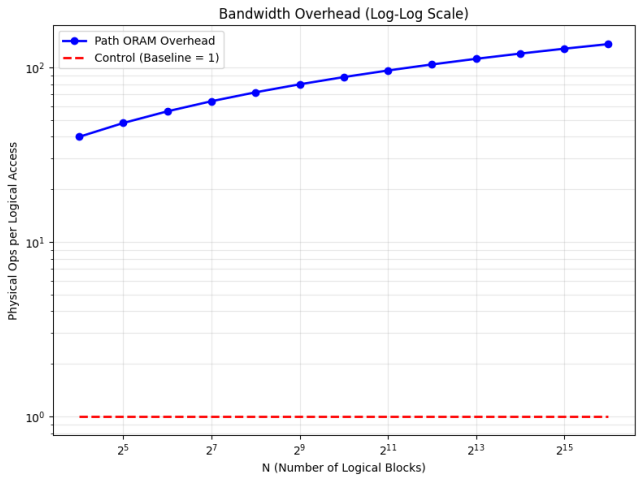


Fig. 2. Bandwidth overhead as a function of logical database size *N*.

From Fig 2. I can confirm that the bandwidth overhead scales logarithmically with *N*, consistent with the theoretical analysis of Path ORAM [3]. While this overhead is significantly higher than non-oblivious memory access, it represents a substantial improvement over early hierarchical ORAM constructions, which incurred polylogarithmic overheads [2], [4].

VI. Limitations and Challenges

Despite strong theoretical guarantees and demonstrated effectiveness in mitigating access-pattern leakage, ORAM remains difficult to deploy widely in practical systems. This section discusses the key limitations and open challenges that continue to hinder the adoption of ORAM in real-world applications.

A. Performance Overhead

The most significant limitation of ORAM is its inherent performance cost. Even state-of-the-art tree-based constructions such as Path ORAM incur logarithmic bandwidth and latency overheads per logical memory access [3]. As demonstrated in Section V, each logical access results in multiple physical memory operations, leading to increased execution time and reduced throughput. These overheads are particularly problematic for latency-sensitive applications such as secure processors, real-time systems, and interactive workloads. Early hierarchical ORAM schemes further exacerbate this issue, exhibiting polylogarithmic overheads that make them impractical for large memory sizes [2], [4]. While modern designs substantially improve efficiency, ORAM remains in orders of magnitude slower than non-oblivious memory access in software-only implementations.

B. Hardware Complexity and Cost

Hardware-assisted ORAM has been proposed to reduce performance overhead by integrating ORAM operations directly into secure processors or memory controllers [20], [21]. These designs increase hardware complexity. They also require more silicon area and consume more power. As a result, ORAM is difficult to implement in standard processors and is mostly limited to specialized high-security systems. Hardware implementations must also manage performance and security separately. Important features for these systems such as caching and speculative execution can have unintentional side-effects into creating side-channel vulnerabilities if they are not controlled with caution.

C. Open Research Challenges

Several open challenges remain in ORAM research. Some of them include reducing overhead further without sacrificing security, developing adaptive ORAM schemes tailored to specific workloads, improving integration with modern processor architectures, and designing systems that provide end-to-end obliviousness across storage, memory, and computation. Addressing these challenges is essential for making ORAM a practical foundation for secure and privacy-preserving systems [4], [15], [21].

VII. Conclusion

ORAM addresses a fundamental limitation of conventional memory protection mechanisms: the leakage of sensitive information through memory access patterns. As demonstrated throughout this survey, encrypting data content alone is insufficient in adversarial environments where memory accesses, timing behavior, and microarchitectural events are observable. ORAM provides a principled solution by ensuring that physical memory access patterns are independent of a program's logical behavior.

This paper presented a comprehensive overview of ORAM, beginning with formal threat models and security definitions and tracing the evolution of classical constructions from hierarchical designs to modern tree-based schemes such as Path ORAM. I examined the role of ORAM in contemporary systems, including trusted execution environments, secure processors, and privacy-preserving computation frameworks, emphasizing both its effectiveness and practical limitations. To complement the survey, I implemented and evaluated a functional Path ORAM prototype, empirically demonstrating access-pattern indistinguishability and quantifying the associated bandwidth and latency overheads. Additionally, the Open Research Challenges I discussed in Section 5. C. elaborate more into what the field has been lacking in terms of more active and practical work.

However, even with significant progress with ORAM and its uses including custom derivatives, it still remains challenging to deploy at scale due to hardware complexity and energy costs. These challenges are a blocker and there is a need for continued research into optimized ORAM designs for designs aiming for oblivious memory systems. As different types of computing platforms evolve like cloud computing, secure enclaves, and privacy-sensitive applications continue to proliferate, ORAM and related oblivious computation techniques are likely to play an increasingly important role in the design of secure and trustworthy systems.

REFERENCES

[1] O. Goldreich and R. Ostrovsky, “Software Protection and Simulation on Oblivious RAMs,” Journal of the ACM, vol. 43, no. 3, pp. 431–473, May 1996.

[2] K. G. Larsen and J. B. Nielsen, “Yes, There is an Oblivious RAM Lower Bound!,” IACR Cryptology ePrint Archive, Paper 2018/423, 2018.

[3] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, and S. Devadas, “Path ORAM: An Extremely Simple Oblivious RAM Protocol,” *in Proc. ACM Conf. Computer and Communications Security (CCS)*, 2013, pp. 299–310.

[4] G. Asharov, I. Komargodski, W. Lin, K. Nayak, and E. Shi, “OptORAMa: Optimal Oblivious RAM,” *in Proc. EUROCRYPT,* 2018, pp. 403–432.

[5] E. Stefanov, E. Shi, and D. Song, "Towards Practical Oblivious RAM," in *Proceedings of the NDSS Symposium 2017*, San Diego, CA, USA, Feb. 2017.

[6] E. Thießen and J. Vahrenhold, "Optimal Offline ORAM with Perfect Security via Simple Oblivious Priority Queues," in *Proceedings of the 35th International Symposium on Algorithms and Computation (ISAAC 2024)*, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Dec. 2024.

[7] G. Asharov et al., “FutORAMa: A New ORAM Construction,” *IACR Cryptology ePrint Archive, Report 2018/123,* 2018.

[8] R. Ostrovsky and V. Shoup, “Private Information Storage,” *in Proc. ACM Symp. Theory of Computing (STOC),* 1997, pp. 294–303.

[9] C. Fletcher, L. Ren, A. Kwon, M. van Dijk, and S. Devadas, “Freecursive ORAM: [Nearly] Free Recursion and Integrity Verification for Position-based Oblivious RAM,” *in Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2015, pp. 103–116.

[10] B. Pinkas and T. Reinman, “Oblivious RAM Revisited,” *in Proc. CRYPTO*, 2010, pp. 502–519.

[11] X. Wang, K. Nayak, C. Liu, E. Shi, E. Stefanov, and Y. Huang, "Oblivious Data Structures," presented at the *35th IEEE Symposium on Security and Privacy*, San Jose, CA, USA, May 2014.

[12] L. Holzbaur, "Data Integrity and Privacy in Distributed Storage," Ph.D. dissertation, *Fakultät für Elektrotechnik und Informationstechnik, Technische Universität München,* Munich, Germany, 2021.

[13] E. D. J. Bernstein, “Cache-timing attacks on AES,” *Department of Mathematics, Statistics, and Computer Science, University of Illinois at Chicago*, Chicago, IL, USA, Apr. 14, 2005.

[14] Y. Xu, W. Cui, and M. Peinado, “Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems,” *in Proc. IEEE Symp. Security and Privacy*, 2015, pp. 640–656.

[15] O. Ohrimenko, M. Costa, C. Fournet, C. Gkantsidis, M. Kohlweiss, and D. Sharma, “Oblivious Multi-Party Machine Learning on Trusted Processors,” *in Proc. USENIX Security Symp.*, 2016, pp. 619–636.

[16] X. S. Wang, K. Nayak, C. Liu, T.-H. H. Chan, E. Shi, E. Stefanov, and Y. Huang, “Oblivious Data Structures,” *IACR Cryptology ePrint Archive*, Report 2014/185, 2014.

[17]] L. Ren, X. Yu, S. Fletcher, M. van Dijk, and S. Devadas, “Design Space Exploration and Optimization of Path Oblivious RAM in Secure Processors,” *in Proc. Int. Symp. Computer Architecture (ISCA),* 2013, pp. 571–582.

[18] Xu, Jian & Zhang, Yuanjing & Fu, Kuiyuan & Peng, Su. (2019). *SGX-based Secure Indexing System. IEEE Access*. PP. 1-1. 10.1109/ACCESS.2019.2921223.

[19] S. Sasy, S. Gorbunov, and C. W. Fletcher, “ZeroTrace: Oblivious Memory Primitives from Intel SGX,” *in Proc. Network and Distributed System Security Symp. (NDSS),* 2018.

[20] M. Maas, E. Love, E. Stefanov, M. Tiwari, E. Shi, K. Asanović, J. Kubiatowicz, and D. Song, “PHANTOM: Practical Oblivious Computation in a Secure Processor,” in Proc. ACM CCS, 2013, pp. 311–324.

[21] C. W. Fletcher, L. Ren, A. Kwon, M. van Dijk, and S. Devadas, “Secure Processor Architecture for Encrypted Computation on Untrusted Memory,” IEEE Micro, vol. 35, no. 5, pp. 1–11, 2015.

[22] R. Canetti, B. Riva, and G. Rothblum, “Practical Delegation of Computation Using Multiple Servers,” in Proc. ACM CCS, 2011, pp. 445–454.